Woxsen University

School of Technology

B.Tech – Computer Science and Engineering

PROJECT BASED LEARNING REPORT

Title of the Project

(Design and Implementation of a Database Management System for Airline Reservation and Flight Operations)

Course Code: 25TU3MJM0

Course Name: Database Management Systems

Semester: 3

Academic Year: 2026–27

Submitted by:

Harshit Jain – 25WU0102094

Section: AIML RHINOS

Under the Guidance of:

Faculty Name: Prof. Sahiti

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Project titled “Design and Implementation of a Database Management System for Airline Reservation and Flight Operations” is a bona fide work carried out by Mr. Harshit Jain (Register No: 25WU0102094), B. Tech – Computer Science and Engineering, during the Academic Year 2026–27, under the supervision and guidance of the faculty mentioned above.

The report presents the conceptual design, logical design and normalization, physical implementation, query and view demonstration, three-tier application architecture, defensive error handling, and testing of the Airline Reservation and Flight Operations database system.

Guide Signature: _____

Prof. Sahiti

DECLARATION

I hereby declare that the Project titled “Design and Implementation of a Database Management System for Airline Reservation and Flight Operations” submitted by me is an original work carried out under the guidance of the faculty mentioned above.

I confirm that this work has not been submitted in part or full for the award of any degree, diploma, or other academic qualification in this or any other institution.

All sources of information used in this report have been properly acknowledged. The technical description is based on the design, schema, queries, triggers and application code developed for this project.

I further declare that the report presents implementation and testing evidence only where such evidence is available in the project work.

Place: Woxsen University

Date:

Signature: _____
Name: Harshit Jain
(25WU0102094)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Woxsen University, School of Technology, and the Department of Computer Science and Engineering for providing the academic environment required for this Project Based Learning work.

I am grateful to my faculty guide, Prof. Sahiti, for guidance, feedback, and encouragement during the design, implementation and documentation of the Airline Reservation and Flight Operations database system. The project combines an Entity-Relationship model, a normalized relational schema, database triggers, SQL queries and views, and a three-tier application into a single working system.

I also acknowledge the standard database textbooks and the MySQL and Flask documentation listed in the References, which formed the technical basis for the design and implementation.

ABSTRACT

Airlines depend on accurate, shared and instantly available data. Passenger details, flight schedules, aircraft allocation, seat inventory, payments and cancellations must all agree with one another, otherwise passengers are double-booked, aircraft are scheduled twice and revenue records drift away from reality. This report presents the design and implementation of a relational Database Management System (DBMS) for Airline Reservation and Flight Operations that removes these problems by storing all operational data in one centralised, constraint-enforced database.

The work begins with the problem analysis and an Entity-Relationship (ER) model covering thirteen entities: Airport, Route, Flight, Aircraft, Seat, Passenger, Booking, Ticket, Payment, Cancellation, Baggage, Check-in and Flight Manifest. The conceptual model is then converted into a relational schema and refined through a documented normalization pipeline (UNF to 1NF to 2NF to 3NF), supported by a structured data dictionary and a list of functional dependencies. The implemented core schema has nine tables, with the remaining ER entities mapped as described in Chapter 4.

Business rules are enforced inside the database through four triggers: aircraft schedule overlap, seat or bed lock, payment validation and automatic booking confirmation. The design is demonstrated with fourteen SQL queries (joins, subqueries and aggregations) and five views. A three-tier application (web client, Flask REST API, MySQL) implements the full workflow of passenger intake, booking, verification, seat allocation and billing, protected by four layers of defensive error handling. A ten-point test matrix and a set of viva-style questions complete the report.

Keywords: airline reservation, relational database, normalization, 3NF, triggers, views, MySQL, three-tier architecture, defensive programming.

TABLE OF CONTENTS

CERTIFICATE....2
DECLARATION....3
ACKNOWLEDGEMENT....4
ABSTRACT....5
TABLE OF CONTENTS....6
LIST OF FIGURES....7
LIST OF TABLES....8
LIST OF ACRONYMS....9
CHAPTER 1: INTRODUCTION....10
1.1 Background and Context....10
1.2 Motivation and Real-World Relevance....10
1.3 Scope and Organization....10
CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES....11
2.1 Problem Definition....11
2.2 Objectives....11
2.3 System Users and Stakeholders....11
2.4 Functional Requirements....12
2.5 Non-Functional Requirements....12
CHAPTER 3: LITERATURE REVIEW / BACKGROUND....13
3.1 Foundations and Related Work....13
3.2 Relational Modelling and Normalization....13
3.3 Triggers, Transactions and Three-Tier Design....13
3.4 Design Position....14
CHAPTER 4: METHODOLOGY & SYSTEM DESIGN....15
4.1 Overall System Architecture....15
4.2 Conceptual Design: ER Diagram....15
4.3 Entities and Relationships....17
4.4 Mapping the ER Model to the Relational Schema....18
4.5 Normalization Pipeline....18
4.6 Functional Dependencies and Data Dictionary....19
CHAPTER 5: IMPLEMENTATION....22
5.1 Environment....22
5.2 Table Definitions (DDL)....22
5.3 Sample Data and Extension Tables....23
5.4 Triggers....24
5.5 Query and View Demonstration....25
5.6 Three-Tier Application Implementation....28
5.7 Defensive Error Handling....30
CHAPTER 6: RESULTS & DISCUSSION....31
6.1 Ten-Point Test Execution Matrix....31
6.2 Execution Procedure....31
6.3 Viva Defence Notes....31
CHAPTER 7: CONCLUSION & FUTURE SCOPE....33
7.1 Future Scope....33
REFERENCES....34

LIST OF FIGURES

Figure 4.2(a): ER diagram, Airline Reservation and Flight Operations Management System..... 16

LIST OF TABLES

Table 2.1(a): Problems addressed by the project.....11
Table 2.3(a): Stakeholders and access needs.....11
Table 3.4(a): Techniques Used and Where They Are Applied.....14
Table 4.1(a): The three tiers.....15
Table 4.3(a): Entities in the ER model.....17
Table 4.3(b): Relationships and cardinalities.....17
Table 4.4(a): Mapping of ER entities to tables.....18
Table 4.5(a): Normalization pipeline summary.....19
Table 4.6(a): Functional dependencies of the 3NF schema.....19
Table 4.6(b): Data dictionary of all tables.....19
Table 5.4(a): Summary of triggers.....25
Table 5.5(a): Views and their users.....28
Table 5.6(a): API endpoints.....28
Table 5.6(b): CRUD operations by workflow stage.....29
Table 5.7(a): Layers of defence.....30
Table 5.7(b): Error handling demonstration.....30
Table 6.1(a): Test execution matrix.....31

LIST OF ACRONYMS

API	Application Programming Interface
BCNF	Boyce–Codd Normal Form
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
ER	Entity-Relationship
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
PNR	Passenger Name Record
REST	Representational State Transfer
SQL	Structured Query Language
UNF	Unnormalized Form
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form

CHAPTER 1 –
INTRODUCTION

1.1 BACKGROUND AND CONTEXT

An airline reservation system is among the most demanding transaction-processing applications. Thousands of passengers search for flights, choose seats and make payments at the same time, while operations staff continuously adjust schedules, aircraft and crews. Every one of these actions reads or changes shared data, and a single inconsistency, such as one seat sold twice, immediately becomes a visible failure at the airport counter.

1.2 MOTIVATION AND REAL-WORLD RELEVANCE

A well-designed database is therefore the foundation of the whole business. It must store data once and correctly, enforce the rules of the airline automatically, answer operational and financial questions quickly, and remain consistent when many users work at the same moment. This project designs such a database from first principles and builds a working application layer around it.

1.3 SCOPE AND ORGANIZATION

- Develop a centralised database for airline reservation and flight operations.
- Manage passenger and customer information efficiently.
- Store and manage flight schedules, routes, airports and aircraft details.
- Manage fares, payments, cancellations and refunds.
- Maintain data integrity and consistency using database constraints.
- Manage seat availability and seat allocation.

The report follows the structure of the Woxsen University Project Based Learning report format. Chapters 1 and 2 establish the problem, objectives and requirements; Chapter 3 gives the background and design position; Chapter 4 describes the methodology and system design, covering the architecture, the ER model, normalization and the data dictionary; Chapter 5 explains the implementation, including the schema, triggers, queries, views, the three-tier application and defensive error handling; Chapter 6 presents testing and discussion; Chapter 7 concludes the report and identifies future scope; and the References section lists the sources used.

CHAPTER 2 –
PROBLEM STATEMENT & OBJECTIVES

2.1 PROBLEM DEFINITION

The motivation for the project comes from three groups of problems that appear when airline data is handled manually or in disconnected files.

Table 2.1(a): Problems addressed by the project

Problem area	Observed issues	Impact
Manual record management	Passenger and flight records are maintained by hand; updates to bookings, passengers and flights are hard to track; records become lost or outdated.	Slow service, inconsistent information, high risk of data loss
Duplicate bookings and seat conflicts	Several passengers may be assigned the same seat; incorrect booking records create allocation conflicts; duplicate bookings occur.	Passenger dissatisfaction, boarding disputes, operational disruption
Baggage tracking problems	Baggage cannot be followed from check-in to arrival; lost or misplaced baggage is not detected early.	Delays, complaints and compensation costs

2.2 OBJECTIVES

- To ensure data accuracy, consistency and integrity using database constraints.
- To manage payments, cancellations and refunds efficiently.
- To design a database that is scalable, secure and easy to maintain.
- To efficiently manage passenger, flight, aircraft and airport information.
- To develop a centralised database for airline reservation and flight operations.

2.3 SYSTEM USERS AND STAKEHOLDERS

Four types of users interact with the system. Each has a different view of the data and a different level of permission.

Table 2.3(a): Stakeholders and access needs

User	Responsibilities	Typical database access
Passenger	Searches flights, makes bookings, selects seats, makes payments and checks in.	Reads flights and own bookings; creates bookings and payments through the application
Reservation Staff	Manages bookings, passenger details, tickets and seat allocation.	Reads and updates passenger, booking, seat and payment data
Flight Operations Staff	Manages flights, aircraft, routes, schedules and flight status.	Reads and updates flight, aircraft, crew and schedule data
Administrator	Manages the database, users, access permissions and system records.	Full privileges; grants and revokes access to views and tables

2.4 FUNCTIONAL REQUIREMENTS

1. The system shall register passengers and prevent duplicate registration by passport number and email.
2. The system shall store airports, aircraft, seat maps, flights and crew assignments.
3. The system shall allow a passenger to search flights by origin, destination and date and see open seats.
4. The system shall create a booking against one passenger, one flight and one seat, and shall never allow a seat to be held twice on the same flight.
5. The system shall never schedule one aircraft on two overlapping flights.
6. The system shall record payments, verify that the amount covers the fare and confirm the booking automatically.
7. The system shall support cancellation, release of the seat and refund status.
8. The system shall provide reports on schedules, itineraries, seat availability, revenue and crew rosters.

2.5 NON-FUNCTIONAL REQUIREMENTS

- **Integrity:** primary keys, foreign keys, unique, check and not-null constraints guard all data.
- **Consistency:** multi-step operations run in transactions and either complete fully or not at all.
- **Security:** parameterised SQL, least-privilege accounts, and role-specific views.
- **Performance:** indexes on foreign keys and search columns keep searches fast.
- **Maintainability:** a 3NF schema with a clear data dictionary so changes touch one place only.
- **Scalability:** a three-tier design lets the web, API and database tiers be scaled separately.

CHAPTER 3 –
LITERATURE REVIEW / BACKGROUND

3.1 FOUNDATIONS AND RELATED WORK

The design in this report rests on established database theory. Entity-Relationship modelling and relational schema design follow the approach described in standard textbooks such as Elmasri and Navathe [1], Silberschatz, Korth and Sudarshan [2], and Date [3]. These sources describe how a real-world situation is captured as entities and relationships, converted into relations with keys and constraints, and refined through normalization to remove redundancy and update anomalies.

The implementation uses MySQL 8 with the InnoDB storage engine [4], which provides transactions, row-level locking and foreign key enforcement, and the Flask framework [5] for the application tier. This report does not survey commercial airline reservation products; it concentrates on the database design techniques described above.

3.2 RELATIONAL MODELLING AND NORMALIZATION

Normalization removes redundancy and the update, insert and delete anomalies that come with it. A relation is in first normal form when it holds only atomic values with no repeating groups. Second normal form removes partial dependencies on part of a composite key, and third normal form removes transitive dependencies between non-key attributes. Boyce–Codd Normal Form (BCNF) strengthens 3NF by requiring that every determinant is a candidate key or superkey. Chapter 4 applies these steps to the airline data, starting from a single unnormalized record.

3.3 TRIGGERS, TRANSACTIONS AND THREE-TIER DESIGN

Constraints such as primary keys, foreign keys, unique, check and not-null declare rules that hold for every row, but some business rules involve several rows or tables, for example two flights of one aircraft overlapping in time. Triggers allow the database itself to enforce such rules at the moment data is inserted or updated, so the rule holds no matter which client sends the change. Transactions group the steps of an operation such as booking and payment so that they either all succeed or are all undone [2][4].

A three-tier architecture separates presentation, application logic and data. The browser never talks directly to the database, and credentials and business rules stay in one controlled place that can be changed or scaled independently.

3.4 DESIGN POSITION

This project combines the classical techniques above in a single end-to-end case study: ER modelling, a documented normalization pipeline to 3NF, business rules implemented as database triggers, a set of queries and views, and a three-tier application with layered error handling, all checked with a ten-point test matrix. The table below maps each technique to the section where it is applied.

Table 3.4(a): Techniques Used and Where They Are Applied

Technique	Purpose	Applied In
ER modelling	Capture entities, attributes and cardinalities	Sections 4.2 – 4.4
Normalization	Remove redundancy and anomalies	Sections 4.5 – 4.6
Triggers	Enforce business rules inside the database	Section 5.4
Queries and views	Joins, subqueries, aggregation and role-specific access	Section 5.5
Three-tier application	Controlled access through a REST API	Sections 4.1, 5.6
Defensive error handling	Reject bad input at every layer	Section 5.7
Test matrix	Verify constraints, triggers and workflow	Section 6.1

CHAPTER 4 –
METHODOLOGY & SYSTEM DESIGN

4.1 OVERALL SYSTEM ARCHITECTURE

The application layer follows the classic three-tier pattern. The browser never talks to the database: every request passes through the application tier, where it is validated and mapped to safe SQL.

Table 4.1(a): The three tiers

Tier 1: Presentation	Tier 2: Application logic	Tier 3: Data
HTML, CSS and JavaScript client	Flask REST API (Python)	MySQL 8 database
Passenger intake and booking forms, seat map, payment screens	Routing, validation, business rules, transaction control, error mapping	9 tables in 3NF, 4 triggers, 5 views, constraints
Client-side checks for quick feedback only	Parameterised SQL only	ACID transactions through InnoDB

Separating the tiers keeps credentials on the server, puts business rules in one place and lets each tier be changed or scaled independently.

4.2 CONCEPTUAL DESIGN: ER DIAGRAM

Figure 4.2(a) shows the Entity-Relationship diagram of the Airline Reservation and Flight Operations Management System. It identifies the main entities, their attributes, the primary keys and the cardinality of each relationship, and it serves as the blueprint for the relational schema.

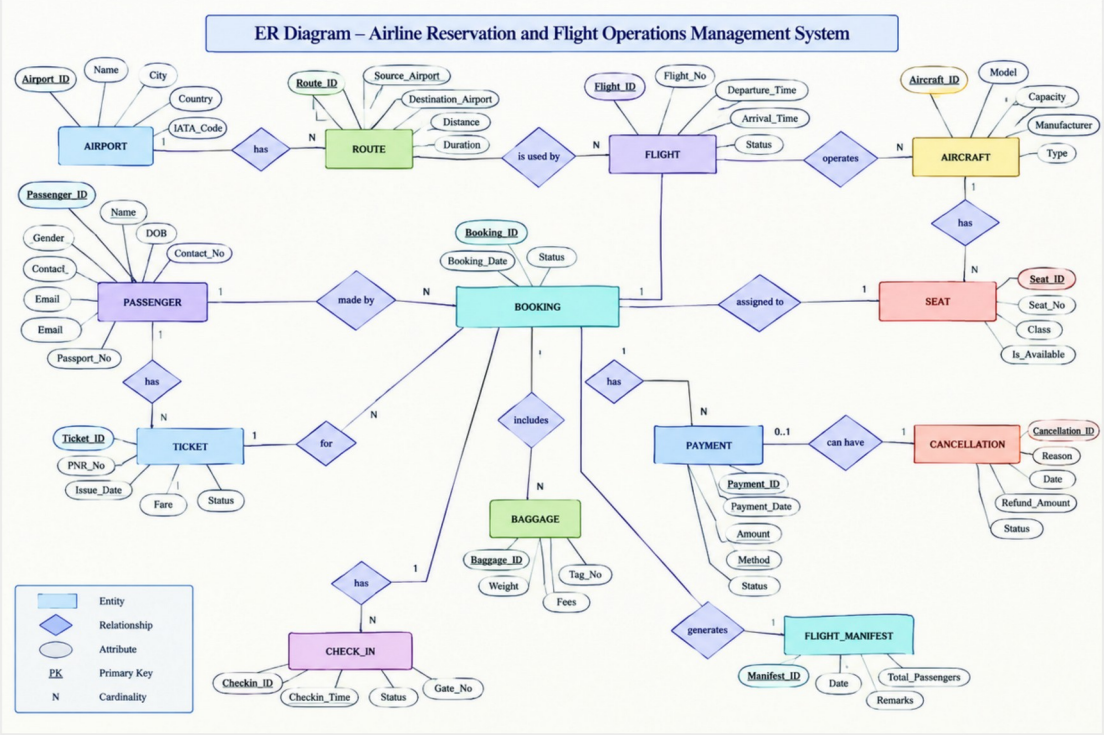


Figure 4.2(a): *ER diagram, Airline Reservation and Flight Operations Management System*

The diagram has seven useful properties. It shows the structure of the system; it shows the relationships between entities; it shows how flights connect to routes, airports and aircraft; it displays the keys used to join tables; it states cardinalities such as one-to-one and one-to-many; it provides a clear blueprint for conversion into a relational database; and it helps to maintain integrity and reduce redundancy.

4.3 ENTITIES AND RELATIONSHIPS

Table 4.3(a): Entities in the ER model

Entity	Key attributes	Description
Airport	Airport_ID, Name, City, Country, IATA_Code	Physical airports served by the airline
Route	Route_ID, Source, Destination, Distance, Duration	Origin-destination pair used by flights
Flight	Flight_ID, Flight_No, Departure, Arrival, Status	A scheduled operation of an aircraft on a route
Aircraft	Aircraft_ID, Model, Capacity, Manufacturer, Type	Fleet vehicles that operate flights
Seat	Seat_ID, Seat_No, Class, Is_Available	Individual seats belonging to an aircraft
Passenger	Passenger_ID, Name, Gender, DOB, Contact, Email, Passport_No	People who travel
Booking	Booking_ID, Booking_Date, Status	A reservation of a seat on a flight by a passenger
Ticket	Ticket_ID, PNR_No, Issue_Date, Fare, Status	Travel document issued for a booking
Payment	Payment_ID, Payment_Date, Amount, Method, Status	Money received for a booking
Cancellation	Cancellation_ID, Reason, Date, Refund_Amount, Status	Cancellation of a paid booking
Baggage	Baggage_ID, Weight, Fees, Tag_No	Checked baggage included in a booking
Check_In	Checkin_ID, Checkin_Time, Status, Gate_No	Airport check-in of a booking
Flight_Manifest	Manifest_ID, Date, Total_Passengers, Remarks	Passenger list generated for a flight

The relationships and their cardinalities, read from the diagram, are:

Table 4.3(b): Relationships and cardinalities

Relationship	Entities	Cardinality
has	Airport - Route	1 : N
is used by	Route - Flight	1 : N (Flight side N)
operates	Aircraft - Flight	1 : N
has	Aircraft - Seat	1 : N
made by	Passenger - Booking	1 : N
assigned to	Booking - Seat	N : 1 (each booking holds one seat)
has	Passenger - Ticket	1 : N
for	Ticket - Booking	Each ticket is for one booking
includes	Booking - Baggage	1 : N
has	Booking - Check_In	1 : N
has	Booking - Payment	1 : N
can have	Payment - Cancellation	0..1 : 1
generates	Flight - Flight_Manifest	1 : 1 per flight

4.4 MAPPING THE ER MODEL TO THE RELATIONAL SCHEMA

The implemented core schema has nine tables. To keep the design in third normal form and avoid redundant tables, some ER entities are represented in a simpler form, and the operational extensions are provided as optional add-on tables. The mapping is summarised below.

Table 4.4(a): Mapping of ER entities to tables

ER entity	Relational representation	Status
Airport	airport (airport_code is the IATA code)	Implemented
Route	origin_code and dest_code columns of flight, each a foreign key to airport	Implemented (merged into flight)
Flight, Aircraft, Seat	flight, aircraft, seat	Implemented
Passenger	passenger	Implemented
Booking, Ticket	booking; a CONFIRMED booking is the issued ticket (booking_id acts as the PNR)	Implemented (merged)
Payment	payment	Implemented
Cancellation	booking.status = CANCELLED plus payment.status = REFUNDED	Implemented through status values
Crew (operations)	crew and flight_crew	Implemented (operations extension)
Baggage, Check_In, Flight_Manifest	Optional extension tables (Section 5.3)	Extension module

4.5 NORMALIZATION PIPELINE

Normalization removes redundancy and the update, insert and delete anomalies that come with it. The design started from a single wide record and was refined in three steps.

Unnormalized form (UNF)

All facts about a booking were first written in one record: booking, passenger, flight, origin and destination airports, aircraft, seat, payment and a list of crew members.

```
AIRLINE_RECORD(booking_id, passenger_name, email, phone, passport_no,
  flight_no, origin_code, origin_city, dest_code, dest_city, dep_time,
  aircraft_model, total_seats, seat_no, class, payment_amount,
  payment_method, crew_list)
```

Problems: crew_list is multi-valued; an airport city is repeated in every booking; changing a passenger email means editing many rows.

First normal form (1NF)

Every attribute must be atomic and there must be no repeating groups. The crew_list was split so that each row holds one crew member, and the composite key (booking_id, crew_id) was chosen. All columns are now single-valued.

Second normal form (2NF)

In 1NF with the key (booking_id, crew_id), flight details depend only on booking_id and crew name depends only on crew_id. These partial dependencies were removed by extracting PASSENGER, FLIGHT, CREW, FLIGHT_CREW and BOOKING into their own relations.

Third normal form (3NF)

Transitive dependencies were removed: airport_code determines city and country (airport table); aircraft_id determines model and total seats (aircraft table); seat_id determines seat number and class (seat table); and booking_id determines the payment facts (payment table).

Table 4.5(a): Normalization pipeline summary

Stage	Rule applied	Result
UNF	None	One wide record with a repeating crew list and heavy redundancy
1NF	Atomic values, no repeating groups	Crew split to one row per member; key chosen
2NF	No partial dependency on part of a composite key	PASSENGER, FLIGHT, CREW, FLIGHT_CREW, BOOKING separated
3NF	No transitive dependency	AIRPORT, AIRCRAFT, SEAT, PAYMENT separated; nine relations

4.6 FUNCTIONAL DEPENDENCIES AND DATA DICTIONARY

Table 4.6(a): Functional dependencies of the 3NF schema

Relation	Functional dependencies
AIRPORT	airport_code -> name, city, country
AIRCRAFT	aircraft_id -> model, total_seats, status
SEAT	seat_id -> aircraft_id, seat_no, class; (aircraft_id, seat_no) -> seat_id
FLIGHT	flight_id -> flight_no, origin_code, dest_code, aircraft_id, dep_time, arr_time, base_fare, status
PASSENGER	passenger_id -> full_name, email, phone, passport_no, dob; email -> passenger_id; passport_no -> passenger_id
BOOKING	booking_id -> passenger_id, flight_id, seat_id, booking_date, status, fare
PAYMENT	payment_id -> booking_id, amount, method, status, paid_at
CREW	crew_id -> name, role, license_no
FLIGHT_CREW	(flight_id, crew_id) -> assigned_role

In every relation each determinant is a candidate key or a superkey, so the schema satisfies 3NF and also Boyce-Codd Normal Form. Derived facts such as city, aircraft model and seat class are stored once and reached through joins.

Table 4.6(b): Data dictionary of all tables

Table	Column	Type	Constraint	Description
airport	airport_code	CHAR(3)	PRIMARY KEY	IATA airport code
airport	name	VARCHAR(100)	NOT NULL	Airport name

Table	Column	Type	Constraint	Description
airport	city	VARCHAR(60)	NOT NULL	City served
airport	country	VARCHAR(60)	NOT NULL	Country
aircraft	aircraft_id	INT	PK, AUTO_INCREMENT	Aircraft identifier
aircraft	model	VARCHAR(50)	NOT NULL	Aircraft model
aircraft	total_seats	INT	NOT NULL, CHECK > 0	Seating capacity
aircraft	status	ENUM	DEFAULT ACTIVE	ACTIVE, MAINTENANCE, RETIRED
seat	seat_id	INT	PK, AUTO_INCREMENT	Seat identifier
seat	aircraft_id	INT	FK -> aircraft, NOT NULL	Owning aircraft
seat	seat_no	VARCHAR(4)	NOT NULL	Seat label, for example 12A
seat	class	ENUM	NOT NULL	ECO, BUS, FIRST
seat	(aircraft_id, seat_no)		UNIQUE	One label per aircraft
flight	flight_id	INT	PK, AUTO_INCREMENT	Flight identifier
flight	flight_no	VARCHAR(8)	NOT NULL	Published flight number
flight	origin_code	CHAR(3)	FK -> airport	Departure airport
flight	dest_code	CHAR(3)	FK -> airport	Arrival airport
flight	aircraft_id	INT	FK -> aircraft	Operating aircraft
flight	dep_time / arr_time	DATETIME	NOT NULL, arr > dep	Schedule
flight	base_fare	DECIMAL(10,2)	CHECK >= 0	Economy base fare
flight	status	ENUM	DEFAULT SCHEDULED	SCHEDULED, DELAYED, DEPARTED, ARRIVED, CANCELLED
passenger	passenger_id	INT	PK, AUTO_INCREMENT	Passenger identifier
passenger	full_name	VARCHAR(100)	NOT NULL	Name
passenger	email	VARCHAR(100)	UNIQUE, NOT NULL	Contact email
passenger	phone	VARCHAR(15)	NOT NULL	Contact number
passenger	passport_no	VARCHAR(15)	UNIQUE, NOT NULL	Passport number
passenger	dob	DATE	NOT NULL	Date of birth
booking	booking_id	INT	PK, AUTO_INCREMENT	Booking reference (PNR)
booking	passenger_id	INT	FK -> passenger	Who booked
booking	flight_id	INT	FK -> flight	Flight booked
booking	seat_id	INT	FK -> seat	Seat held
booking	booking_date	DATETIME	DEFAULT NOW	Time of booking
booking	status	ENUM	DEFAULT HOLD	HOLD, CONFIRMED, CANCELLED
booking	fare	DECIMAL(10,2)	NOT NULL, CHECK >= 0	Fare charged
payment	payment_id	INT	PK, AUTO_INCREMENT	Payment identifier
payment	booking_id	INT	FK -> booking	Booking paid for
payment	amount	DECIMAL(10,2)	NOT NULL, CHECK > 0	Amount paid
payment	method	ENUM	NOT NULL	CARD, UPI, NETBANKING, CASH

Table	Column	Type	Constraint	Description
payment	status	ENUM	NOT NULL	PAID, FAILED, REFUNDED
payment	paid_at	DATETIME	DEFAULT NOW	Payment time
crew	crew_id	INT	PK, AUTO_INCREMENT	Crew identifier
crew	name	VARCHAR(100)	NOT NULL	Crew member name
crew	role	ENUM	NOT NULL	PILOT, CABIN
crew	license_no	VARCHAR(20)	UNIQUE, NOT NULL	Licence number
flight_crew	flight_id	INT	PK part, FK -> flight	Flight
flight_crew	crew_id	INT	PK part, FK -> crew	Crew member
flight_crew	assigned_role	VARCHAR(20)	NOT NULL	Role on this flight

CHAPTER 5 –
IMPLEMENTATION

5.1 ENVIRONMENT

The database is implemented in MySQL 8 with the InnoDB storage engine, which provides transactions, row-level locking and foreign key enforcement. Character set utf8mb4 is used so that names in any script are stored correctly. All SQL in this chapter can be run in order from a MySQL client.

5.2 TABLE DEFINITIONS (DDL)

```
CREATE DATABASE airline_db CHARACTER SET utf8mb4;
USE airline_db;

CREATE TABLE airport (
  airport_code CHAR(3) PRIMARY KEY,
  name    VARCHAR(100) NOT NULL,
  city    VARCHAR(60)  NOT NULL,
  country VARCHAR(60)  NOT NULL
);

CREATE TABLE aircraft (
  aircraft_id INT AUTO_INCREMENT PRIMARY KEY,
  model       VARCHAR(50) NOT NULL,
  total_seats INT NOT NULL CHECK (total_seats > 0),
  status ENUM('ACTIVE','MAINTENANCE','RETIRED') NOT NULL DEFAULT 'ACTIVE'
);

CREATE TABLE seat (
  seat_id     INT AUTO_INCREMENT PRIMARY KEY,
  aircraft_id INT NOT NULL,
  seat_no     VARCHAR(4) NOT NULL,
  class ENUM('ECO','BUS','FIRST') NOT NULL,
  UNIQUE (aircraft_id, seat_no),
  FOREIGN KEY (aircraft_id) REFERENCES aircraft(aircraft_id)
);

CREATE TABLE flight (
  flight_id   INT AUTO_INCREMENT PRIMARY KEY,
  flight_no   VARCHAR(8) NOT NULL,
  origin_code CHAR(3) NOT NULL,
  dest_code   CHAR(3) NOT NULL,
  aircraft_id INT NOT NULL,
  dep_time    DATETIME NOT NULL,
  arr_time    DATETIME NOT NULL,
  base_fare   DECIMAL(10,2) NOT NULL CHECK (base_fare >= 0),
  status ENUM('SCHEDULED','DELAYED','DEPARTED','ARRIVED','CANCELLED')
         NOT NULL DEFAULT 'SCHEDULED',
  CHECK (arr_time > dep_time),
  CHECK (origin_code <> dest_code),
  FOREIGN KEY (origin_code) REFERENCES airport(airport_code),
  FOREIGN KEY (dest_code)   REFERENCES airport(airport_code),
  FOREIGN KEY (aircraft_id) REFERENCES aircraft(aircraft_id)
);

CREATE TABLE passenger (
  passenger_id INT AUTO_INCREMENT PRIMARY KEY,
  full_name   VARCHAR(100) NOT NULL,
  email       VARCHAR(100) NOT NULL UNIQUE,
  phone       VARCHAR(15)  NOT NULL,
  passport_no VARCHAR(15)  NOT NULL UNIQUE,
  dob         DATE NOT NULL
);

CREATE TABLE booking (
  booking_id   INT AUTO_INCREMENT PRIMARY KEY,
  passenger_id INT NOT NULL,
  flight_id    INT NOT NULL,
  seat_id      INT NOT NULL,
  booking_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  status ENUM('HOLD','CONFIRMED','CANCELLED') NOT NULL DEFAULT 'HOLD',
  fare   DECIMAL(10,2) NOT NULL CHECK (fare >= 0),
  FOREIGN KEY (passenger_id) REFERENCES passenger(passenger_id),
  FOREIGN KEY (flight_id)    REFERENCES flight(flight_id),
  FOREIGN KEY (seat_id)      REFERENCES seat(seat_id)
);

CREATE TABLE payment (
  payment_id INT AUTO_INCREMENT PRIMARY KEY,
  booking_id INT NOT NULL,
  amount     DECIMAL(10,2) NOT NULL CHECK (amount > 0),
  method ENUM('CARD','UPI','NETBANKING','CASH') NOT NULL,
  status ENUM('PAID','FAILED','REFUNDED') NOT NULL,
  paid_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (booking_id) REFERENCES booking(booking_id)
);

CREATE TABLE crew (
  crew_id INT AUTO_INCREMENT PRIMARY KEY,
  name    VARCHAR(100) NOT NULL,
  role    ENUM('PILOT','CABIN') NOT NULL,
  license_no VARCHAR(20) NOT NULL UNIQUE
);

CREATE TABLE flight_crew (
  flight_id INT NOT NULL,
  crew_id   INT NOT NULL,
  assigned_role VARCHAR(20) NOT NULL,
  PRIMARY KEY (flight_id, crew_id),
  FOREIGN KEY (flight_id) REFERENCES flight(flight_id),
  FOREIGN KEY (crew_id)   REFERENCES crew(crew_id)
);

CREATE INDEX idx_booking_flight ON booking(flight_id, seat_id);
CREATE INDEX idx_flight_route   ON flight(origin_code, dest_code, dep_time);
```

Foreign keys use the default RESTRICT action, so a passenger or flight that still has bookings cannot be deleted. Cancelled bookings are kept as history by changing their status instead of deleting rows.

5.3 SAMPLE DATA AND EXTENSION TABLES

A small demonstration data set is loaded so that every query in Section 5.5 returns visible results. Aircraft are given six seats each to keep the data compact.

```
INSERT INTO airport VALUES
 ('HYD','Rajiv Gandhi International','Hyderabad','India'),
 ('DEL','Indira Gandhi International','Delhi','India'),
 ('BOM','Chhatrapati Shivaji Maharaj International','Mumbai','India');

INSERT INTO aircraft VALUES (1,'Airbus A320',6,'ACTIVE'),
                            (2,'Boeing 737-800',6,'ACTIVE');

INSERT INTO seat (seat_id,aircraft_id,seat_no,class) VALUES
 (1,1,'1A','BUS'),(2,1,'1B','BUS'),(3,1,'2A','ECO'),
 (4,1,'2B','ECO'),(5,1,'2C','ECO'),(6,1,'2D','ECO'),
 (7,2,'1A','BUS'),(8,2,'1B','BUS'),(9,2,'2A','ECO'),
 (10,2,'2B','ECO'),(11,2,'2C','ECO'),(12,2,'2D','ECO');

INSERT INTO flight VALUES
 (101,'AI201','HYD','DEL',1,'2026-11-10 06:00','2026-11-10 08:30',7000,'SCHEDULED'),
 (102,'AI202','DEL','HYD',1,'2026-11-10 10:00','2026-11-10 12:30',7200,'SCHEDULED'),
 (103,'6E301','HYD','BOM',2,'2026-11-10 07:00','2026-11-10 08:45',4500,'SCHEDULED'),
 (104,'AI205','HYD','DEL',2,'2026-11-11 06:00','2026-11-11 08:30',5000,'SCHEDULED');

INSERT INTO passenger VALUES
 (1,'Aarav Mehta','aarav@mail.com','9876500001','P1234567','1998-04-12'),
 (2,'Sneha Rao','sneha@mail.com','9876500002','P2345678','1995-09-30'),
 (3,'Rohan Gupta','rohan@mail.com','9876500003','P3456789','2000-01-18'),
 (4,'Isha Nair','isha@mail.com','9876500004','P4567890','1992-06-05');

INSERT INTO booking VALUES
 (1,1,101,1,'2026-10-01 10:00','CONFIRMED',21000),
 (2,2,101,3,'2026-10-02 11:00','CONFIRMED',7000),
```

```
 (3,3,103,9,'2026-10-03 12:00','HOLD',4500),
 (4,1,104,8,'2026-10-05 09:00','CONFIRMED',15000);

INSERT INTO payment VALUES
 (1,1,21000,'CARD','PAID','2026-10-01 10:05'),
 (2,2,7000,'UPI','PAID','2026-10-02 11:03'),
 (3,4,15000,'CARD','PAID','2026-10-05 09:02');

INSERT INTO crew VALUES (1,'Capt. Vikram Singh','PILOT','LIC-1001'),
 (2,'Anita Das','CABIN','LIC-2001'),(3,'Capt. Meera Iyer','PILOT','LIC-1002');
INSERT INTO flight_crew VALUES (101,1,'CAPTAIN'),(101,2,'PURSER'),(103,3,'CAPTAIN');
```

The optional extension tables below add the remaining ER entities. They reference the core tables and are in 3NF.

```
CREATE TABLE baggage (
  baggage_id INT AUTO_INCREMENT PRIMARY KEY,
  booking_id INT NOT NULL,
  tag_no     VARCHAR(12) NOT NULL UNIQUE,
  weight_kg  DECIMAL(5,2) NOT NULL CHECK (weight_kg > 0),
  fees       DECIMAL(8,2) NOT NULL DEFAULT 0,
  FOREIGN KEY (booking_id) REFERENCES booking(booking_id)
);
CREATE TABLE check_in (
  checkin_id INT AUTO_INCREMENT PRIMARY KEY,
  booking_id INT NOT NULL UNIQUE,
  checkin_time DATETIME NOT NULL,
  gate_no VARCHAR(5),
  status ENUM('CHECKED_IN','BOARDED','NO_SHOW') NOT NULL,
  FOREIGN KEY (booking_id) REFERENCES booking(booking_id)
);
CREATE TABLE flight_manifest (
  manifest_id INT AUTO_INCREMENT PRIMARY KEY,
  flight_id INT NOT NULL UNIQUE,
  manifest_date DATE NOT NULL,
  total_passengers INT NOT NULL,
  remarks VARCHAR(200),
  FOREIGN KEY (flight_id) REFERENCES flight(flight_id)
);
```

5.4 TRIGGERS

Triggers enforce business rules inside the database, so the rules hold for every client, whether it is the web application, an administrator script or a manual SQL session. In MySQL these are row-level triggers that fire on INSERT; they are data-manipulation triggers, not DDL triggers. Each rejects an invalid row with SQLSTATE 45000 and a clear message.

Trigger 1: Aircraft schedule overlap

An aircraft cannot be in two places at once. Two time intervals overlap when each starts before the other ends.

```
DELIMITER $$
CREATE TRIGGER trg_flight_overlap BEFORE INSERT ON flight
FOR EACH ROW
BEGIN
  IF EXISTS (SELECT 1 FROM flight
             WHERE aircraft_id = NEW.aircraft_id
               AND NEW.dep_time < arr_time
               AND NEW.arr_time > dep_time) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Aircraft schedule overlap';
  END IF;
END$$
DELIMITER ;
```

Trigger 2: Seat or bed lock

A seat, or a lie-flat bed seat in premium cabins, can be held by only one active booking per flight. Cancelled bookings release the seat.

```
DELIMITER $$
CREATE TRIGGER trg_seat_lock BEFORE INSERT ON booking
FOR EACH ROW
BEGIN
  IF EXISTS (SELECT 1 FROM booking
             WHERE flight_id = NEW.flight_id AND seat_id = NEW.seat_id
               AND status <> 'CANCELLED') THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Seat already locked';
  END IF;
END$$
DELIMITER ;
```

Triggers 3 and 4: Payment validation and booking confirmation

```
DELIMITER $$
CREATE TRIGGER trg_payment_check BEFORE INSERT ON payment
FOR EACH ROW
BEGIN
  DECLARE v_fare DECIMAL(10,2);
  SELECT fare INTO v_fare FROM booking WHERE booking_id = NEW.booking_id;
  IF NEW.status = 'PAID' AND NEW.amount < v_fare THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Payment below fare';
  END IF;
END$$

CREATE TRIGGER trg_payment_confirm AFTER INSERT ON payment
FOR EACH ROW
BEGIN
  IF NEW.status = 'PAID' THEN
    UPDATE booking SET status = 'CONFIRMED'
    WHERE booking_id = NEW.booking_id;
  END IF;
END$$
DELIMITER ;
```

Table 5.4(a): Summary of triggers

Trigger	Event	Rule enforced
trg_flight_overlap	BEFORE INSERT on flight	No aircraft double-scheduling
trg_seat_lock	BEFORE INSERT on booking	One active booking per seat per flight
trg_payment_check	BEFORE INSERT on payment	A paid amount must cover the fare
trg_payment_confirm	AFTER INSERT on payment	Paid booking becomes CONFIRMED automatically

5.5 QUERY AND VIEW DEMONSTRATION

Fourteen queries are grouped by technique. Each is run on the sample data of Section 5.3.

JOIN QUERIES

Q1: Passenger itineraries (three-table inner join)

```
SELECT p.full_name, f.flight_no, f.dep_time, b.status
FROM booking b
JOIN passenger p USING (passenger_id)
JOIN flight f    USING (flight_id);
```

Lists who is travelling on which flight and the state of each booking.

Q2: Routes with city names (airport joined twice)

```
SELECT f.flight_no, o.city AS origin, d.city AS destination
FROM flight f
JOIN airport o ON f.origin_code = o.airport_code
JOIN airport d ON f.dest_code   = d.airport_code;
```

The airport table is joined twice, once for each end of the route.

Q3: Crew roster per flight

```
SELECT f.flight_no, c.name, c.role
FROM flight_crew fc
JOIN crew c   USING (crew_id)
JOIN flight f USING (flight_id)
ORDER BY f.flight_no;
```

Q4: Bookings with no payment (LEFT JOIN)

```
SELECT b.booking_id, p.full_name
FROM booking b
JOIN passenger p USING (passenger_id)
LEFT JOIN payment y USING (booking_id)
WHERE y.payment_id IS NULL;
```

On the sample data this returns booking 3, which is still on HOLD.

Q5: Seat map of flight 101 with occupancy

```
SELECT s.seat_no, s.class, b.booking_id
FROM flight f
JOIN seat s ON s.aircraft_id = f.aircraft_id
LEFT JOIN booking b ON b.seat_id = s.seat_id AND b.flight_id = f.flight_id
WHERE f.flight_id = 101;
```

Seats with a NULL booking_id are free.

SUBQUERIES

Q6: Flights priced above the average fare (scalar subquery)

```
SELECT flight_no, base_fare FROM flight
WHERE base_fare > (SELECT AVG(base_fare) FROM flight);
```

Q7: Passengers who have never booked (NOT IN)

```
SELECT full_name FROM passenger
WHERE passenger_id NOT IN (SELECT passenger_id FROM booking);
```

Returns Isha Nair in the sample data.

Q8: Busiest flight by number of bookings

```
SELECT flight_no FROM flight
WHERE flight_id = (SELECT flight_id FROM booking
                   GROUP BY flight_id ORDER BY COUNT(*) DESC LIMIT 1);
```

Q9: Passengers with a payment above 20,000 (correlated EXISTS)

```
SELECT full_name FROM passenger p
WHERE EXISTS (SELECT 1 FROM booking b
              JOIN payment y USING (booking_id)
              WHERE b.passenger_id = p.passenger_id AND y.amount > 20000);
```

AGGREGATE QUERIES

Q10: Revenue per flight

```
SELECT b.flight_id, SUM(y.amount) AS revenue
FROM booking b JOIN payment y USING (booking_id)
WHERE y.status = 'PAID'
GROUP BY b.flight_id;
```

Q11: Load factor (percentage of seats sold)

```
SELECT f.flight_no,
       ROUND(COUNT(b.booking_id) * 100 / a.total_seats, 1) AS load_pct
```

```
FROM flight f
JOIN aircraft a USING (aircraft_id)
LEFT JOIN booking b ON b.flight_id = f.flight_id AND b.status <> 'CANCELLED'
GROUP BY f.flight_id, f.flight_no, a.total_seats;
```

Q12: Bookings by cabin class

```
SELECT s.class, COUNT(*) AS bookings
FROM booking b JOIN seat s USING (seat_id)
GROUP BY s.class;
```

Q13: Routes operated at least twice (HAVING)

```
SELECT origin_code, dest_code, COUNT(*) AS trips
FROM flight
GROUP BY origin_code, dest_code
HAVING COUNT(*) >= 2;
```

HYD to DEL appears for flights 101 and 104.

Q14: Monthly revenue trend

```
SELECT DATE_FORMAT(paid_at, '%Y-%m') AS month, SUM(amount) AS revenue
FROM payment
WHERE status = 'PAID'
GROUP BY month
ORDER BY month;
```

VIEWS

Views hide joins, simplify reporting and allow each role to see only the columns it needs. Five views are defined.

```
CREATE VIEW v_flight_schedule AS
SELECT f.flight_id, f.flight_no, o.city AS origin, d.city AS destination,
       f.dep_time, f.arr_time, a.model, f.status
FROM flight f
JOIN airport o ON f.origin_code = o.airport_code
JOIN airport d ON f.dest_code   = d.airport_code
JOIN aircraft a USING (aircraft_id);

CREATE VIEW v_passenger_itinerary AS
SELECT b.booking_id, p.full_name, f.flight_no, s.seat_no, s.class,
       f.dep_time, b.status
FROM booking b
JOIN passenger p USING (passenger_id)
JOIN flight f    USING (flight_id)
JOIN seat s      USING (seat_id);

CREATE VIEW v_seat_availability AS
SELECT f.flight_id, s.seat_no, s.class,
       IF(b.booking_id IS NULL, 'AVAILABLE', 'LOCKED') AS state
FROM flight f
JOIN seat s USING (aircraft_id)
LEFT JOIN booking b ON b.seat_id = s.seat_id
     AND b.flight_id = f.flight_id AND b.status <> 'CANCELLED';

CREATE VIEW v_revenue_summary AS
SELECT f.flight_id, f.flight_no,
       COALESCE(SUM(CASE WHEN y.status='PAID' THEN y.amount END),0) AS revenue,
       COUNT(DISTINCT b.booking_id) AS bookings,
       ROUND(COUNT(DISTINCT b.booking_id)*100/a.total_seats,1) AS load_pct
FROM flight f
JOIN aircraft a USING (aircraft_id)
LEFT JOIN booking b ON b.flight_id = f.flight_id AND b.status <> 'CANCELLED'
LEFT JOIN payment y ON y.booking_id = b.booking_id
GROUP BY f.flight_id, f.flight_no, a.total_seats;

CREATE VIEW v_crew_roster AS
SELECT f.flight_no, c.name, c.role, fc.assigned_role
FROM flight_crew fc
JOIN crew c   USING (crew_id)
JOIN flight f USING (flight_id);
```

Table 5.5(a): Views and their users

View	Purpose	Used by
v_flight_schedule	Flights with city names, aircraft model and status	Operations desk
v_passenger_itinerary	Passenger, flight, seat and booking status in one row	Customer service
v_seat_availability	Seats per flight marked AVAILABLE or LOCKED	Booking engine
v_revenue_summary	Paid revenue, bookings and load factor per flight	Finance
v_crew_roster	Crew names, roles and assigned flights	Crew planning

5.6 THREE-TIER APPLICATION IMPLEMENTATION

REST API

Table 5.6(a): API endpoints

Method	Endpoint	Purpose	Tables used
POST	/api/passengers	Register passenger	passenger
GET	/api/flights?from=&to=&date=	Search flights with open seats	flight, airport, v_seat_availability
POST	/api/bookings	Create booking (HOLD)	booking
PUT	/api/bookings/{id}/seat	Allocate or change seat	booking, seat
POST	/api/payments	Pay and confirm booking	payment, booking
DELETE	/api/bookings/{id}	Cancel booking, release seat	booking

A request follows five steps: the client sends JSON; the API validates types, ranges and required fields; the API opens a transaction and runs parameterised SQL; triggers and constraints enforce the database rules; finally the API commits or rolls back and maps the result to an HTTP status code.

CRUD WORKFLOW: INTAKE, BOOKING, VERIFICATION, SEAT AND BILLING

The end-to-end workflow has five stages. Each stage performs create, read, update and delete operations on its own data. The verification stage covers passport, fare and eligibility checks before a seat is allocated.

Table 5.6(b): CRUD operations by workflow stage

Stage	Create	Read	Update	Delete
1. Intake	INSERT passenger	Find by passport or email	Edit contact details	Delete only if no bookings
2. Booking	INSERT booking (HOLD)	Search flights with open seats	Change flight, same fare class	Cancel: status = CANCELLED
3. Verification	Record document check	View check status	Mark VERIFIED or REJECTED	Void failed check
4. Seat / Bed	Assign seat to booking	v_seat_availability	Move to another seat	Release seat on cancel
5. Billing	INSERT payment (trigger validates)	v_revenue_summary, invoice	Booking becomes CONFIRMED; refund status	Void unpaid HOLD

IMPLEMENTATION SAMPLE

The booking endpoint below shows the pattern used by every write operation: validate, read trusted values from the database, write inside a transaction, and translate errors into clean HTTP responses.

```
@app.post("/api/bookings")
def create_booking():
    d = request.get_json(silent=True) or {}
    missing = [k for k in ("passenger_id","flight_id","seat_id") if k not in d]
    if missing:
        return err(400, f"Missing fields: {missing}")
    cn = pool.get_connection()
    try:
        cn.start_transaction()
        cur = cn.cursor()
        cur.execute("SELECT base_fare FROM flight WHERE flight_id=%s",
                    (d["flight_id"],))
        row = cur.fetchone()
        if row is None:
            return err(404, "Flight not found")
        cur.execute("INSERT INTO booking (passenger_id,flight_id,seat_id,"
                    "fare,status) VALUES (%s,%s,%s,%s,'HOLD')",
                    (d["passenger_id"], d["flight_id"], d["seat_id"], row[0]))
        cn.commit()
        return ok(201, {"booking_id": cur.lastrowid})
    except mysql.connector.Error as e:
        cn.rollback()
        return err(409 if e.sqlstate == "45000" else 500, e.msg)
    finally:
        cn.close()
```

- Placeholders (%s) keep SQL and data separate, which prevents SQL injection.
- The fare is read from the database and never trusted from the client.
- A trigger error (45000) becomes HTTP 409, so the user sees a clear message.
- Rollback leaves no half-written booking, and the connection is always released.

5.7 DEFENSIVE ERROR HANDLING

Defensive design assumes that every input may be wrong and every component may fail. The system therefore uses four layers of protection, so that a mistake missed by one layer is caught by the next.

Table 5.7(a): Layers of defence

Layer	Protection provided
Client	Required fields, formats, date and number ranges. This gives fast feedback but is never trusted.
API	Schema checks, type coercion, allow-listed values and safe error messages that never expose stack traces.
Transaction	All-or-nothing writes; ROLLBACK on any exception; connection closed in a finally block.
Database	NOT NULL, UNIQUE, FOREIGN KEY, CHECK constraints and the four triggers form the last line of defence.

The table below lists the demonstration cases used to show the defences working. For each case the request is sent, the response is shown, and the tables are then checked to confirm that nothing changed.

Table 5.7(b): Error handling demonstration

Bad input	Caught by	HTTP	Message returned
Booking with missing seat_id	API validation	400	Missing fields: [seat_id]
Unknown flight_id = 9999	API existence check	404	Flight not found
Same seat booked twice	trg_seat_lock	409	Seat already locked
Aircraft in overlapping flights	trg_flight_overlap	409	Aircraft schedule overlap
Payment lower than fare	trg_payment_check	422	Payment below fare
Duplicate passport number	UNIQUE constraint	409	Passenger already registered
Delete passenger who has bookings	FOREIGN KEY	409	Cannot delete: bookings exist
Search text: ' OR 1=1 --	Parameterised query	200	0 results, no data leak

Principles followed: validate early, enforce centrally, fail safely and never expose internals. Errors are logged on the server with full detail, while the client receives only a short, safe message and a suitable status code.

CHAPTER 6 –
RESULTS & DISCUSSION

6.1 TEN-POINT TEST EXECUTION MATRIX

The matrix covers creation, constraints, triggers, workflow completion, referential integrity and security. The Status column is left empty so that the result of each run (Pass or Fail) can be recorded when the tests are executed.

Table 6.1(a): Test execution matrix

#	Test case	Input / action	Expected result	Status
1	Register passenger	Valid name, email and passport	201; row created	
2	Duplicate passport	Re-insert the same passport	409; no new row	
3	Valid booking	Free seat on flight 101	201; status HOLD	
4	Seat lock	Book the same seat again	409 Seat already locked	
5	Aircraft overlap	Insert flight with overlapping times	Rejected by trigger	
6	Underpayment	Amount below fare	422; no payment row	
7	Valid payment	Amount equals fare	Payment saved; booking CONFIRMED	
8	Cancel booking	DELETE /bookings/{id}	CANCELLED; seat available again	
9	Referential integrity	Delete passenger with bookings	409; passenger kept	
10	SQL injection	' OR 1=1 -- in search box	Treated as text; 0 rows	

6.2 EXECUTION PROCEDURE

1. Create the database from Section 5.2 and load the sample data of Section 5.3.
2. Create the triggers and views.
3. Start the API and run tests 1 to 10 from the client or with curl.
4. After each test, query the affected table to confirm the expected state.
5. Record Pass or Fail in the matrix and note the actual message returned.

6.3 VIVA DEFENCE NOTES

The following questions summarise the main design decisions and the answers that justify them.

Q1. Why a three-tier architecture instead of letting the client talk to MySQL?
Security, because no database credentials reach the browser; one central place for business rules; and independent scaling and maintenance of each tier.

Q2. Why normalise to 3NF?

It removes redundancy and the update, insert and delete anomalies. Airport, aircraft and seat details are stored once and joined when needed.

Q3. What is the difference between 3NF and BCNF?

3NF allows a dependency whose right side is a prime attribute; BCNF requires every determinant to be a superkey. The schema satisfies both.

Q4. Why use views?

They hide joins, simplify reports and let each role see only the columns it needs.

Q5. Which indexes help most?

Foreign keys, a composite index on booking (flight_id, seat_id) for seat checks, and flight (origin_code, dest_code, dep_time) for searches.

Q6. Why use triggers when the API already validates?

Triggers protect the data even if another client or script bypasses the API; the database stays correct on its own.

Q7. How are two simultaneous bookings of one seat prevented?

The seat-lock trigger inside a transaction. For stricter safety, add SELECT ... FOR UPDATE on the seat row or a unique key on active seat bookings.

Q8. Explain ACID in the booking flow.

Atomic: booking and payment succeed together or not at all. Consistent: constraints always hold. Isolated: concurrent bookings do not interfere. Durable: committed rows survive a crash.

Q9. What happens if payment fails after the seat is held?

The transaction rolls back or the booking stays on HOLD, and a scheduled job cancels expired holds to free the seat.

Q10. How is SQL injection prevented?

All queries use bound parameters, input is validated, and the database account has least privilege.

CHAPTER 7 –
CONCLUSION & FUTURE SCOPE

The project delivers a systematic and efficient airline reservation and flight operations database. Its main achievements are:

- A systematic and efficient airline reservation management system.
- Reduced manual record-keeping and human error.
- Prevention of duplicate bookings and seat conflicts through the seat-lock trigger and constraints.
- Improved passenger and baggage tracking through the baggage and check-in extension.
- Better coordination between passengers, staff and administrators through role-specific views.
- Secure and organised storage of reservation data.
- Faster and easier booking, cancellation and flight management.
- Improved overall customer experience and operational efficiency.

Technically, the design moves from an ER model with thirteen entities to a nine-table 3NF core schema, enforces its rules with four triggers, demonstrates its use with fourteen queries and five views, and wraps it in a three-tier application with four layers of defensive error handling and a ten-point test matrix.

7.1 FUTURE SCOPE

- Dynamic pricing and fare classes with seasonal rules.
- A loyalty programme with points earned per flight.
- Full baggage tracking with scan events at every handling point.

REFERENCES

[1] Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.

[2] Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts, McGraw-Hill.

[3] Date, C. J., An Introduction to Database Systems, Pearson.

[4] Oracle Corporation, MySQL 8.0 Reference Manual (triggers, views, InnoDB transactions).

[5] Pallets Projects, Flask Documentation.

— End of Report —